Lección 08: Web Scraping - Extracción de Datos de la Web 🤖🌐

El Web Scraping es el proceso automatizado de extraer información de sitios web. Es una técnica poderosa para recopilar datos, monitorear precios, generar leads y mucho más. En esta lección, aprenderemos a hacerlo de tres maneras: con un método básico, con una herramienta robusta y, finalmente, con una técnica avanzada para sitios modernos.

Parte 1: El Enfoque Básico (y Frágil) con Expresiones Regulares

01: Scraping con requests y Regex

La forma más directa de empezar a "scrapear" es descargar el contenido HTML de una página como un gran bloque de texto y luego usar Expresiones Regulares (que ya aprendimos) para buscar los patrones que nos interesan.
Paso 1: Descargar el HTML. Usamos la librería requests para hacer una petición GET a la URL y obtener todo su código fuente HTML como una cadena de texto.

```
import requests

url = 'https://www.apple.com/es/shop/buy-mac/macbook-air/'
response = requests.get(url)

if response.status_code == 200:
  print('La petición fue exitosa. HTML descargado.')
  html = response.text
else:
  print('La petición falló.')
```

Paso 2: Extraer Datos con Regex. Una vez que tenemos el HTML, definimos un patrón de Regex para encontrar la información que queremos, como el título de la página o el precio de un producto, y usamos re.search() para encontrarlo.

```
import re

# (Suponiendo que 'html' contiene el código fuente de la página)

# Patrón para encontrar el contenido de la etiqueta <title>
title_pattern = r'<title>(.*?)</title>'
match_title = re.search(title_pattern, html)
if match_title:
  print(f"El título de la web es: {match_title.group(1)}")

# Patrón para encontrar un precio dentro de un <span> específico
```

```
price_pattern = r'<span class="rc-prices-fullprice">(.*?)</span>'
match_price = re.search(price_pattern, html)
if match_price:
  print(f"El precio del producto es: {match_price.group(1)}")
```

¿Por qué es un método frágil? Este enfoque depende totalmente de la estructura exacta del HTML. Si el sitio web cambia ligeramente (por ejemplo, modifica el nombre de una clase de rc-prices-fullprice a rc-price-main), nuestra expresión regular dejará de funcionar. Por eso, necesitamos una herramienta más inteligente.

Parte 2: Parsing Robusto de HTML con BeautifulSoup

02: Introducción a BeautifulSoup

BeautifulSoup es una librería de Python diseñada para "parsear" (analizar y comprender) documentos HTML y XML. En lugar de ver el HTML como un simple texto, lo convierte en un "árbol" de objetos que podemos navegar y buscar de forma mucho más flexible y robusta.
Paso 1: Crear la "Sopa". Primero, instalamos la librería (pip install beautifulsoup4). Luego, le pasamos nuestro texto HTML para crear un objeto BeautifulSoup, que cariñosamente llamamos "sopa".

```
from bs4 import BeautifulSoup
import requests

url = 'https://www.apple.com/es/shop/buy-mac/macbook-air/'
# Es buena práctica incluir un User-Agent para identificarte.
headers = {
  'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124
Safari/537.36'
}
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, 'html.parser')
```

Paso 2: Encontrando Elementos. BeautifulSoup nos ofrece métodos muy intuitivos para buscar en el árbol HTML:

- soup.find('tag', class_='class_name'): Encuentra la **primera** etiqueta que coincide.
- soup.find_all('tag', class_='class_name'): Encuentra **todas** las etiquetas que coinciden y las devuelve en una lista.

<!-- end list -->

```
# Encontrar la etiqueta del título y obtener su texto
title_tag = soup.title
if title_tag:
    print(f"El título de la web es: {title_tag.text}")

# Encontrar todos los productos y, para cada uno, buscar su nombre y
precio
products = soup.find_all(class_='rc-productselection-item')
```

```
for product in products:
    # Dentro de cada producto, buscamos la etiqueta del nombre y la
del precio
    name = product.find(class_="list-title").text.strip()
    price = product.find(class_="rc-prices-fullprice").text.strip()
    print(f"Producto: {name}\nPrecio: {price}\n")
```

03: Proyecto - Herramienta SEO en Línea de Comandos

Ahora, combinemos BeautifulSoup con el módulo argparse para crear una herramienta de línea de comandos (CLI) que realice una auditoría SEO básica en cualquier URL.
Paso 1: Aceptando Argumentos con argparse. argparse es la forma estándar en Python de crear interfaces de línea de comandos. Definimos los argumentos que nuestro script espera recibir, en este caso, una url.

```
import argparse

# Configura el parser para aceptar un argumento posicional llamado
'url'
parser = argparse.ArgumentParser(description="Web scraping para
chequeo SEO.")
parser.add_argument('url', type=str, help='La URL del sitio que
quieres analizar.')
args = parser.parse_args()
url = args.url

# Para ejecutarlo desde la terminal: python tu_script.py
https://ejemplo.com
```

Paso 2: La Lógica de Auditoría SEO. Usamos BeautifulSoup para verificar algunas reglas básicas de SEO: ¿la página tiene un título?, ¿es demasiado largo?, ¿hay exactamente una etiqueta <h1>?

```
# (Suponiendo que ya tenemos el objeto 'soup' de la URL)

print(f"\nRevisando la página: {url}")

# Chequeo del título
titulo_pagina = soup.title.string
if titulo_pagina:
    print(f"El título de la página es: {titulo_pagina}")
    if len(titulo_pagina) < 70:
      print("✅  El título tiene una longitud adecuada.")
    else:
      print("⚠️  El título es DEMASIADO largo.")
else:
    print("❌  No se encontró etiqueta <title>.")

# Chequeo de la etiqueta H1
```

```
titulos_h1 = soup.find_all('h1')
if not titulos_h1:
    print("❌  No se encontraron títulos <h1> en la página.")
elif len(titulos_h1) > 1:
    print("❌  Hay más de un título <h1> en la página.")
else:
    print("✅  Hay exactamente un título <h1> en la página.")
```

Parte 3: Scraping Avanzado de Sitios Dinámicos con Playwright

04: Scraping e Interacción con Playwright

El problema de los sitios modernos: Muchas páginas web de hoy en día cargan su contenido dinámicamente usando JavaScript. requests y BeautifulSoup solo ven el HTML inicial, que a menudo está vacío, y se pierden todo el contenido que JavaScript añade después.
La solución: Playwright. Es una herramienta de "automatización de navegadores". En lugar de solo descargar el HTML, lanza un navegador web real (como Chrome), que ejecuta el JavaScript, espera a que todo el contenido cargue, e incluso puede interactuar con la página (hacer clics, rellenar formularios). Esto nos permite scrapear la página final tal y como la ve un usuario.
Paso 1: Lanzar un Navegador y Navegar. Tras instalarlo (pip install playwright y playwright install), el código para iniciar es muy sencillo. El bloque with se encarga de abrir y cerrar el navegador automáticamente.

```
from playwright.sync_api import sync_playwright

url = 'https://midu.dev'

with sync_playwright() as p:
  # Lanza un navegador Chromium. headless=False nos permite ver lo que
pasa.
  browser = p.chromium.launch(headless=False, slow_mo=500)
  page = browser.new_page()
  page.goto(url)

  print("Página cargada. Ahora podemos interactuar con ella.")
  # ... aquí iría el código de scraping ...

  browser.close()
```

Paso 2: Localizar Elementos e Interactuar. Playwright usa "localizadores" para encontrar elementos en la página. Una vez localizado, podemos obtener su contenido o realizar acciones como hacer clic.

```
# (Dentro del bloque 'with' anterior)
```

```
# page.locator('article a') encuentra todos los enlaces <a> dentro de
un <article>
# .first se queda con el primero de ellos.
first_article_anchor = page.locator('article a').first

# Obtenemos el texto del enlace
article_title = first_article_anchor.text_content()
print(f"El título del primer artículo es: {article_title}")

# Hacemos clic en el enlace para navegar a la página del artículo
print("Haciendo clic en el enlace...")
first_article_anchor.click()

# Esperamos a que la nueva página termine de cargar
page.wait_for_load_state('networkidle')
print(f"Navegamos a la página: {page.title()}")
```

Playwright es una herramienta increíblemente potente no solo para el scraping, sino también para realizar tests automatizados de aplicaciones web, asegurando que los enlaces, botones y formularios funcionen como se espera.